

1. Introduction à la notation Grand O (Big O)

Échelle des complexités courantes (de la plus rapide à la plus lente)

-
- Big-O Complexity**
- The graph illustrates the growth of various Big-O time complexities as the number of elements increases from 0 to 100. The Y-axis represents the number of operations, ranging from 0 to 1000. The X-axis represents the number of elements, ranging from 0 to 100.
- The complexities shown are:
- $O(1)$ (Red line): Constant time complexity, showing a flat line near zero.
 - $O(\log n)$ (Dark red line): Logarithmic time complexity, showing a very slow increase.
 - $O(n)$ (Green line): Linear time complexity, showing a steady, linear increase.
 - $O(n \log n)$ (Purple line): Slightly super-linear time complexity, showing a faster increase than linear.
 - $O(n^2)$ (Teal line): Quadratic time complexity, showing a significant increase in operations as the number of elements grows.
 - $O(2^n)$ (Orange line): Exponential time complexity, showing extremely rapid growth.
 - $O(n!)$ (Blue line): Factorial time complexity, showing the fastest growth, reaching 1000 operations at around 7 elements.

2. La Complexité Temporelle et les Structures natives de Python

L'élégance syntaxique de Python cache parfois des réalités algorithmiques lourdes. Il est crucial de savoir ce qui se passe sous le capot.

Piège 1 : L'opérateur `in` (Liste VS Set/Dictionnaire)

Vérifier la présence d'un élément dans une liste oblige Python à la parcourir entièrement ($O(n)$). Dans un dictionnaire ou un `set`, Python utilise une table de hachage ($O(1)$).

```
ma_liste = list(range(10000000))
mon_set = set(ma_liste)

# Recherche dans une liste :  $O(n)$  -> Lent
if 9999999 in ma_liste:
    pass

# Recherche dans un set :  $O(1)$  -> Instantané
if 9999999 in mon_set:
    pass
```

Piège 2 : Modifier le début d'une liste

Les listes Python sont des tableaux dynamiques contigus en mémoire.

```
import random
from collections import deque

l = [random.randint(0,1000000) for _ in range(1000000)]

# Python doit décaler tous les éléments d'une case
l.insert(0, 42)

print(l.pop(0))

# utiliser les collections en adéquation avec les besoins
q = deque([random.randint(0,1000000) for _ in range(1000000)])

# Beaucoup plus rapide
q.append(42)

print(q.pop())
```

2. Recherche linéaire vs dichotomique (dans une liste triée)

1. L'approche naïve : Recherche Linéaire. On inspecte chaque élément un par un.

```
def recherche_lineaire(liste, target):  
    for index, element in enumerate(liste):  
        if element == target:  
            return index  
    return -1
```

On élimine un élément à chaque itération.

Pire des cas (dans une liste de 100 éléments):

$100 - 1 \rightarrow 99$

$99 - 1 \rightarrow 98$

...

$3 - 1 \rightarrow 2$

$2 - 1 \rightarrow 1$

- pour 100 valeurs, on itérera 100 fois
- pour n valeurs, on itérera n fois $\rightarrow O(n)$

2. L'approche Optimisée : Recherche Dichotomique

```
def recherche_dichotomique(liste, target):  
    bas, haut = 0, len(liste) - 1  
    while bas <= haut:  
        milieu = (bas + haut) // 2  
        if liste[milieu] == target:  
            return milieu  
        elif liste[milieu] > target:  
            haut = milieu - 1 # On cherche à gauche  
        else:  
            bas = milieu + 1 # On cherche à droite  
    return -1
```

On élimine la moitié des éléments à chaque itération.

Pire des cas (dans une liste de 100 éléments):

$$100/2 \rightarrow 50$$

$$50/2 \rightarrow 25$$

$$25/2 \rightarrow 12$$

$$12/2 \rightarrow 6$$

$$6/2 \rightarrow 3$$

$$3/2 \rightarrow 1$$

- pour 100 valeurs, on itérera $\log_2 100 \approx 6$ fois
- pour n valeurs, on itérera $\log_2 n$ fois $\rightarrow O(\log n)$

3. Tableau

Taille de la liste (n)	Recherche Linéaire $O(n)$	Recherche Dichotomique $O(\log n)$
10	10 opérations	~4 opérations
1 000	1 000 opérations	~10 opérations
1 000 000	1 000 000 opérations	~20 opérations
1 000 000 000	1 milliard d'opérations	~30 opérations

Remarque:

Le tri d'une liste en Python (.sort()) coûte $O(n \log n)$.

ne trie pas une liste pour n'y faire qu'une seule recherche.

La dichotomie est rentable uniquement si la liste est déjà triée par nature, ou si vous effectuez un très grand nombre de requêtes sur la même structure.

3. Programmation Dynamique

La programmation dynamique consiste à résoudre un problème complexe en le découpant en sous-problèmes, et en stockant leurs résultats pour s'éviter de les recalculer.

Prenons la suite de Fibonacci : $f(n) = f(n - 2) + f(n - 1)$

1. Version naïve (Récursive pure)

```
def fibo_naive(n):
    if(n <= 1):
        return n
    return fibo_naive(n - 2) + fibo_naive(n - 1)
```

ex:

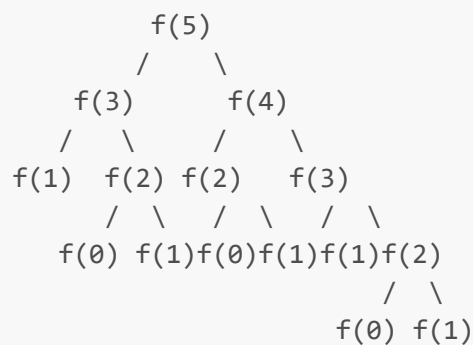
$$f(5) = f(3) + f(4)$$

$$\Rightarrow f(5) = f(1) + f(2) + f(2) + f(3)$$

$$\Rightarrow f(5) = f(1) + f(0) + f(1) + f(0) + f(1) + f(1) + f(2)$$

$$\Rightarrow f(5) = f(1) + f(0) + f(1) + f(0) + f(1) + f(1) + f(0) + f(1)$$

$$\Rightarrow f(5) = 1 + 0 + 1 + 0 + 1 + 1 + 0 + 1 = 5$$



Complexité

- Temporelle : $O(2^n)$ (Fige le script dès $n > 40$)
- Spatiale : $O(n)$

2. Version Récursive avec Mémoïsation (Top-Down)

```
def fibo_cache(n, cache = None):
    if not cache:
        cache = {}
    if(n <= 1):
        return n
    if(n in cache):
        return cache[n]
    cache[n] = fibo_cache(n - 2, cache) + fibo_cache(n - 1, cache)
    return cache[n]
```

ex:

$$f(5) = f(3) + f(4)$$

$$\Rightarrow f(5) = f(3) + f(2) + f(3)$$

$$\Rightarrow \text{mise de cache de } f(2) = f(0) + f(1) = 1$$

$$\Rightarrow \text{mise de cache de } f(3) = f(1) + f(2) = 2$$

$$\Rightarrow f(5) = f(1) + f(0) + f(1) + f(2) + f(3) = 1 + 0 + 1 + 1 + 2 = 5$$



[cache] = Lu directement dans le cache

- Temporelle : $O(n)$ (Chaque nombre n est calculé qu'une seule fois)
- Spatiale : $O(n)$ (Taille du cache + pile d'appels)

Mémoïsation native:

Python propose un décorateur permettant d'automatiser le cache (dans la RAM)

```
from functools import cache

@cache
def fibo_cache_native(n):
    if(n <= 1):
        return n
    return fibo_cache_native(n - 2) + fibo_cache_native(n - 1)
```

3. Version Itérative avec Mémoïsation (Bottom-Up)

On part du bas (0 et 1) et on monte de manière impérative à l'aide d'une boucle et d'un tableau.

```
def fibo_iteratif(n):  
    if n <= 1:  
        return n  
  
    dp = [0] * (n + 1)  
    dp[1] = 1  
  
    for i in range(2, n + 1):  
        dp[i] = dp[i - 1] + dp[i - 2]  
  
    return dp[n]
```

- Temporelle : $O(n)$
- Spatiale : $O(n)$ (Allocation d'un tableau de taille $n + 1$)

4. Version Itérative avec Optimisation Spatiale

```
def fibo_iteratif_no_cache(n):  
    f1, f2 = 0, 1  
    for i in range(2, n + 1):  
        f1, f2 = f2, f1 + f2  
    return 0 if n == 0 else f2
```

- Temporelle : $O(n)$
- Spatiale : $O(1)$ (la complexité spatiale ne dépend pas de n)

4. cProfile

La théorie du Grand O donne la complexité théorique, mais **cProfile** fournit la mesure réelle en production.

C'est le module natif de Python pour identifier précisément les **goulots d'étranglement** (*bottlenecks*) en mesurant la fréquence et la durée d'exécution de chaque fonction.

```
import cProfile
import pstats
profiler = cProfile.Profile()

profiler.enable()
somme = sum(i for i in range(1_000_000))
profiler.disable()

stats = pstats.Stats(profiler).sort_stats('cumulative')
stats.print_stats(10)
```